01 Memory: addresses and values

Start here before you think about pointers.

Every memory location has an address.

An address identifies a location; the value lives at that location.

A memory address is not the data stored there.

Example memory snapshot

Address	Value
x4000	7
x4001	12
x4002	-3

Memory[x4000] = 7

x4000 names the box. The number 7 is stored inside the box.

This distinction drives how LD, LDR, and STR work.

02 Pointers: a register holds an address

An LC-3 register can store a memory location, not just data.

A pointer is a value interpreted as a memory address.

When R2 stores x4000, R2 points to memory location x4000.

R2 = x4000

Register R2		Memory address x4000
x4000	points to	**Value: 7**

R2 contains the address x4000, NOT the number 7.

Dereferencing means reading memory at the stored address.

LDR uses a base register plus a signed offset to calculate an address.

That is why you must understand pointers before LDR.

03 LDR: follow the pointer

Base + offset -> effective address -> read value.

LDR uses a register value to locate data in memory.

Format: LDR DR, BaseR, offset6

```
AND R1, R1, #0
LDR R1, R2, #0
```

LDR R1, R2, #0

Given: R2 = x4000 and Memory[x4000] = 7

Effective address = R2 + 0 = x4000

Load value at x4000 into R1 -> R1 becomes 7.

LDR reads memory at the address stored in a base register.

By contrast, LD uses a PC-relative address, not BaseR.

04 STR: write through the pointer

The matching store instruction reverses the data direction.

STR writes a register value into a memory location.

Format: STR SR, BaseR, offset6

```
STR R1, R2, #0
```

STR R1, R2, #0

Given: R1 = 9 and R2 = x4000

Effective address = R2 + 0 = x4000

Memory[x4000] becomes 9.

LDR: memory -> register

STR: register -> memory

Both use base-plus-offset addressing.